



SERIES #1: CRACK MNC JAVA INTERVIEW

About Series #1: Crack MNC Java Interview

Week 1: Top OOPS Questions for Interview

- 💡 Any interview starts with OOP concepts
- Think of OOPs as the bricks when you're building a house.
- That's why it's crucial to be completely perfect with OOPs before diving into any interview!

Why OOPs is Important in Interviews

- ✓ If an interviewer asks any question related to OOPs, you must be ready to answer confidently.
- ✓ That's why we've dedicated the first week to mastering OOPs concepts.

Case 1: 🎓 Especially for College Students and Experienced Professionals

- Interviewers will focus mainly on the basics of OOPs.
- So, be strong with OOPs!
- Understand the concepts thoroughly.
- Be prepared to explain:
 - How you applied OOPs in your college project
 - How OOPs help in developing real-time applications (this is very important!).

Why First Impressions Matter

- ✨ The first impression is the best impression!
- This is where you set the tone for the entire interview.

👉 We'll provide you with:

- A list of questions.
- A sample answer pattern to help you frame your answers

Adapt this pattern to give structured, confident answers.

Pro Tip:

- 📝 Giving the best answer for OOPs questions increases your chances of clearing the interview by 75%!
- Top OOPs Topics to Cover for Java Interviews
- Before attending any interview, make sure you've mastered

Important Differences-Based OOP FAQs

Difference between method overloading and method overriding

Features	Method overloading	Method overriding
Definition	If a class having multiple methods having same name but different in number of parameter's is known as method overloading.	Method overriding process of inheriting the methods from the superclass and changing its implementation in sub class
Use / Advantages	Both increase the readability of program	used to provide specific implementation for superclass method or To Reuse same method for Different purpose in sub class
Parameters	Parameters must be different to achieve method overloading	Parameters must be Same to achieve Method overriding
Example:	Compile time poly	Run time poly
Rules	- We can't achieve method overloading by changing return type of only. - it causes ambiguity during execution and through error.	- IS-A relationship is mandatory between superclass and subclass - We should retain Super class method Signature in subclass also - Subclass has permission to increase the access level, but subclass doesn't have access to decrease the access level

Important Differences-Based OOP FAQs

Abstract class v/s interface

Similarities:

- both are used to achieve abstraction
- both can't create its objects directly

Differences:

Abstract Class 🟡	Interface 🔵
✅ Can have abstract and non-abstract methods.	❌ Can have only abstract methods (till JDK 7). Since Java 8, can have default and static methods .
❌ Doesn't support multiple inheritance.	✅ Supports multiple inheritance.
✅ Can have final, non-final, static, and non-static variables.	❌ Has only static and final variables by default.
✅ Can provide implementation for an interface.	❌ Cannot provide implementation for abstract classes.
🔧 Uses abstract keyword.	🔧 Uses interface keyword.
✅ Can extend another Java class and implement multiple interfaces.	❌ Can extend another interface only .
🔑 Can be extended using the extends keyword.	🔑 Can be implemented using the implements keyword.
🔒 Members can be private, protected, or public .	🔒 Members are public by default .

Important Differences-Based OOP FAQs

? Are abstraction and abstract class the same?	✗ No, both are different things.
🔍 Abstraction	- 🟢 One of the OOP principles. - 📦 Hides implementation details and shows only the essential information to the end user.
📦 Abstract Class	- 📦 A type of class declared using the abstract keyword. - 💡 One of the ways to implement the idea of abstraction.

Association vs Composition

Concept	Association 📁	Composition 🏠
Definition	A relationship where two objects are connected but independent.	A strong relationship where one object depends on the other.
Dependency	Objects can exist independently.	One object cannot exist without the other.
Lifetime	The lifecycle of associated objects is independent.	The lifecycle of composed objects is tied to the parent object.
Example	A teacher teaches multiple students , and students can learn from different teachers.	A car has an engine —the car cannot function without the engine.

IS-A vs HAS-A Relationship

Concept	IS-A Relationship 🐾	HAS-A Relationship 🔗
Definition	Represents inheritance or a parent-child relationship.	Represents composition or aggregation.
Nature	Indicates a type of relationship (generalization).	Indicates a part-of relationship.
Keywords	extends or implements in Java.	Objects are linked using fields or references.
Example	A dog IS-A animal .	A car HAS-A engine .

Task for readers:

These are the most FAQ OOP basics! 🚀

- **📖 Explore More:** Dive deeper into OOP FAQs, practice, and try small code snippets for each concept 📖. We'll share detailed code snippets on the community channel soon. **For now, we want you to explore and learn!** 🚀
- **🔧 For Experienced Professionals:** Reflect on how you've implemented these concepts in your current project.
- **🎓 For College Students & Graduates:** Relate these concepts to your syllabus and try implementing small projects.

Pro Tip: Think Big, Start Small! 🌟 Let's grow together!

Note:

- This document is meant for brush-ups key concepts & Top FAQ.
- We always recommend working on projects and practicing hands-on coding to solidify your concepts. 📖🚀

Copyright Notice: 📄

🚫 **Personal Use Only:** This content is strictly for personal use.

🚫 **No Unauthorized Sharing:** Do not post on social media (LinkedIn, Instagram, etc.) without permission.

⚖️ **Strict Copyright Action:** If you share this ePDF on your personal Instagram or LinkedIn without permission, your account may be suspended or permanently removed due to copyright strikes ⚠️.

💖 **Share Responsibly:** You can share it privately with friends preparing for interviews, but not on social media 🚫

✉️ Contact for Permissions:

- Email us at: support@javatechcommunity.com
- DM us on Instagram: [@java_tech_community](https://www.instagram.com/java_tech_community)